

Vídeos no Youtube	1
Ligações	1
Códigos	1
Python	1
ESP32	27
Observações sobre os códigos	34
Código extra em python para limpar a memória caso haja muita transferência de dados e lixo de memória	35

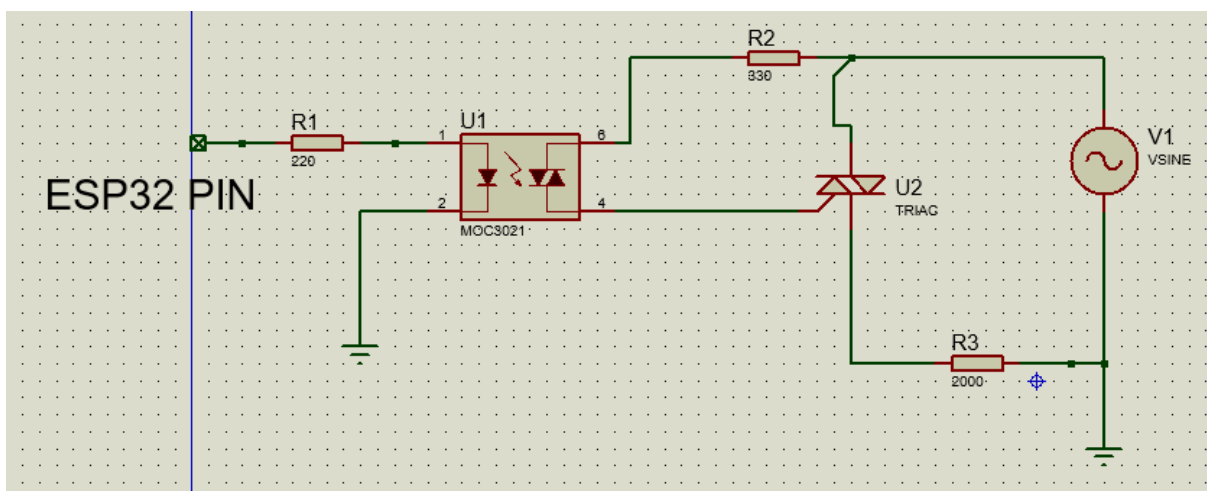
Vídeos no Youtube

<https://youtube.com/shorts/YBSa0W0Evt0>

<https://youtube.com/shorts/8ahPZacc1X8>

<https://youtube.com/shorts/qSBsnuZCngU>

Ligações



Importante dizer que essa configuração funciona em cargas de baixíssima corrente elétrica. Não recomendo usar em cargas maiores, inclusive já queimei componentes por isso. O TRIAC usado foi o BTA 16.

Códigos

Python

```

import serial
import time
import threading
import pygame
from pathlib import Path
import sys

# =====
# CONFIGURAÇÕES PRINCIPAIS
# =====
PORTA_COM = "COM6"
BAUD_RATE = 115200

# Caminhos das músicas (AJUSTE AQUI!)
MUSICAS = {
    "baile_favela": r"C:\Users\eduar\Downloads\Bobby Helms - Jingle Bell Rock.mp3",
    "lady_gaga": r"C:\Users\eduar\Downloads\ladygaga.mp3",
    "avaba": r"C:\Users\eduar\Downloads\25 MINUTOS DE ELETRÔNICAS PESADAS [ 150
BPM ] DJ RANTARO 2K18.mp3",
    "dont_stop": r"C:\Users\eduar\Downloads\Journey - Don't Stop Believin' (Escape Tour
1981_ Live In Houston).mp3"
}

# =====
# CLASSE PRINCIPAL DO ESP32 COM CONFIRMAÇÃO
# =====
class ESP32ShowController:
    """Controlador definitivo para shows de luzes com ESP32"""

    def __init__(self, porta, baud_rate):
        self.porta = porta
        self.baud_rate = baud_rate
        self.conexao = None
        self.conectado = False
        self.ack_habilitado = True
        self.cor_atual = [0, 0, 0, 0, 0] # R, G, B, W, Y

    def conectar(self):
        """Conecta ao ESP32 e configura"""
        try:
            print(f"🔌 Conectando ao ESP32 na porta {self.porta}...")
            self.conexao = serial.Serial(
                port=self.porta,
                baudrate=self.baud_rate,
                timeout=2,
                dsrdtr=False,
                rtscts=False

```

```

)

# Configurações para evitar reset
self.conexao.dtr = False
self.conexao.rts = False

time.sleep(2)
self.conexao.reset_input_buffer()

print("⌚ Aguardando inicialização do ESP32...")
inicio = time.time()

while time.time() - inicio < 10:
    if self.conexao.in_waiting > 0:
        linha = self.conexao.readline().decode(errors='ignore').strip()
        if "READY" in linha:
            print("✅ ESP32 conectado e pronto!")
            self.conectado = True

            # Configura modo com confirmações
            self._enviar_comando_raw("ACK,1", esperar_resposta=True)

            return True
        time.sleep(0.1)

    print("⚠️ ESP32 não respondeu, tentando continuar...")
    self.conectado = True
    return True

except Exception as e:
    print(f"❌ Erro na conexão: {e}")
    return False

def _enviar_comando_raw(self, comando, esperar_resposta=True, timeout=1):
    """Envia comando e processa respostas"""
    if not self.conectado or not self.conexao:
        return False, None

    try:
        if not comando.endswith("\n"):
            comando += '\n'

        self.conexao.write(comando.encode())
        self.conexao.flush()

        if not esperar_resposta:
            return True, None

```

```

# Coleta respostas
todas_respostas = []
resposta_final = None
inicio = time.time()

while time.time() - inicio < timeout:
    if self.conexao.in_waiting > 0:
        linha = self.conexao.readline().decode(errors='ignore').strip()
        if linha:
            todas_respostas.append(linha)

            if linha.startswith("OK:"):
                resposta_final = linha
            elif linha.startswith("ERR:"):
                resposta_final = linha

        if resposta_final:
            break

    time.sleep(0.01)

sucesso = resposta_final is not None and resposta_final.startswith("OK:")
return sucesso, resposta_final

except Exception as e:
    print(f"❌ Erro ao enviar comando: {e}")
    return False, None

# =====
# COMANDOS DE ALTO NÍVEL
# =====
def fade(self, r, g, b, w=0, y=0, tempo_ms=1000):
    """Fade para cor em tempo específico"""
    comando = f"F,{r},{g},{b},{w},{y},{tempo_ms}"
    return self._enviar_comando_raw(comando)[0]

def set_cor(self, r, g, b, w=0, y=0):
    """Define cor dos LEDs com confirmação"""
    comando = f"S,{r},{g},{b},{w},{y}"
    sucesso, resposta = self._enviar_comando_raw(comando)

    if sucesso:
        self.cor_atual = [r, g, b, w, y]

    return sucesso

def wait(self, ms):
    """Espera no ESP32 (mais preciso)"""

```

```

    comando = f"W,{ms}"
    return self._enviar_comando_raw(comando)[0]

def reset(self):
    """Apaga todos os LEDs"""
    return self.set_cor(0, 0, 0, 0, 0)

# =====
# SEQUÊNCIAS PRÉ-PROGRAMADAS
# =====

def sequencia_lady_gaga_simples(self, duracao_total_ms=30000):

    """Versão limpa: matriz de cores → executa uma por uma"""
    print("🎵 Lady Gaga - Versão Limpa")

    # CONFIGURAÇÃO
    tempo_ligado = 100 # ms com luz acesa
    tempo_desligado = 100 # ms com luz apagada

    # MATRIZ DE CORES - uma para cada batida
    # Formato: (R, G, B, W, Y)
    self.wait(400)
    tempo_ligado = 1000 # ms com luz acesa
    tempo_desligado = 1 # ms com luz apagada

    for a in range (2):
        #1
        self.set_cor(255, 0, 0, 0, 0)
        self.wait(tempo_ligado)
        self.set_cor(255, 255, 0, 0, 0)
        self.wait(tempo_ligado)

        self.set_cor(255, 255, 255, 0, 0)
        self.wait(tempo_ligado)
        self.set_cor(255, 255, 255, 255, 0)
        self.wait(tempo_ligado)

        self.set_cor(255, 255, 255, 255, 255)
        self.wait(tempo_ligado)

        self.set_cor(125, 125, 125, 125, 125)
        self.wait(tempo_ligado)

        self.set_cor(255, 255, 255, 0, 0)
        self.wait(tempo_ligado)

```

```

        self.set_cor(0, 0, 0, 255, 255)
        self.wait(tempo_ligado)
# MATRIZ DE CORES - uma para cada batida
# Formato: (R, G, B, W, Y)
self.wait(400)
tempo_ligado = 500 # ms com luz acesa
tempo_desligado = 1 # ms com luz apagada

for a in range (2):
    #1
    self.set_cor(255, 0, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_ligado)

    self.set_cor(255, 255, 255, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 255, 255, 0)
    self.wait(tempo_ligado)

    self.set_cor(255, 255, 255, 255, 255)
    self.wait(tempo_ligado)

    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_ligado)

    self.set_cor(255, 255, 255, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(0, 0, 0, 255, 255)
    self.wait(tempo_ligado)
# MATRIZ DE CORES - uma para cada batida
# Formato: (R, G, B, W, Y)
self.wait(400)
tempo_ligado = 250 # ms com luz acesa
tempo_desligado = 1 # ms com luz apagada
#24s
for a in range (4):
    #1
    self.set_cor(255, 0, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_ligado)

    self.set_cor(255, 255, 255, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 255, 255, 0)
    self.wait(tempo_ligado)

```

```
self.set_cor(255, 255, 255, 255, 255)
self.wait(tempo_ligado)
```

```
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_ligado)
```

```
self.set_cor(255, 255, 255, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 255, 255)
self.wait(tempo_ligado)
```

#32s

tempo_ligado = 100 # ms com luz acesa

tempo_desligado = 100 # ms com luz apagada

for a in range (10):

#1

```
self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

#2

```
self.set_cor(0, 255, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(0, 255, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

#3

```
self.set_cor(0, 0, 255, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(0, 0, 255, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

#4

```
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
#5
```

```
self.set_cor(0, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(0, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
#6
```

```
self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
#7
```

```
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(0, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
#8
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 0, 0)
self.wait(tempo_desligado)
```

```
self.set_cor(0, 0, 0, 0, 255)
```



```

    self.wait(tempo_ligado)
    self.set_cor(0, 0, 0, 0, 0)
    self.wait(tempo_desligado)
#64s
for a in range (10):
    #1
    self.set_cor(255, 0, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)

    self.set_cor(255, 0, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)
    #2
    self.set_cor(0, 255, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)

    self.set_cor(0, 255, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)
    #3
    self.set_cor(0, 0, 255, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)

    self.set_cor(0, 0, 255, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)
    #4
    self.set_cor(0, 0, 0, 255, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)

    self.set_cor(0, 0, 0, 255, 0)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 125, 125, 125)
    self.wait(tempo_desligado)
    #5

    self.set_cor(0, 0, 0, 0, 255)

```

```

self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)

self.set_cor(0, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)
#6
self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)

self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)
#7
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)

self.set_cor(0, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)

#8
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)

self.set_cor(0, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(125, 125, 125, 125, 125)
self.wait(tempo_desligado)
#96s
tempo_ligado = 1000 # ms com luz acesa
tempo_desligado = 1 # ms com luz apagada

for a in range (3):
    #1
    self.fade(255, 0, 0, 0, 0, 1000)
    self.wait(tempo_ligado)
    self.fade(255, 255, 0, 0, 0, 1000)

```

```

self.wait(tempo_ligado)

self.fade(255, 255, 255, 0, 0, 1000)
self.wait(tempo_ligado)
self.fade(255, 255, 255, 255, 0, 1000)
self.wait(tempo_ligado)

self.fade(255, 255, 255, 255, 255, 1000)
self.wait(tempo_ligado)

self.fade(125, 125, 125, 125, 125, 1000)
self.wait(tempo_ligado)

self.fade(0, 0, 0, 255, 255, 1000)
self.wait(tempo_ligado)
self.fade(255, 255, 0, 0, 0, 1000)
self.wait(tempo_ligado)

# Fade final
self.fade(0, 0, 0, 0, 0, 2000)
def sequencia_dont_stop_simples(self, duracao_total_ms=30000):

    """Versão limpa: matriz de cores → executa uma por uma"""
    print("🎵 Lady Gaga - Versão Limpa")

    # CONFIGURAÇÃO
    tempo_ligado = 500 # ms com luz acesa
    tempo_desligado = 500 # ms com luz apagada

    # MATRIZ DE CORES - uma para cada batida
    # Formato: (R, G, B, W, Y)
    for b in range(4):
        for a in range(2):#4s vezes 2
            #1
            self.set_cor(255, 0, 0, 0, 0)
            self.wait(tempo_ligado)
            self.set_cor(255, 255, 0, 0, 0)
            self.wait(tempo_ligado)

            self.set_cor(255, 255, 255, 0, 0)
            self.wait(tempo_ligado)
            self.set_cor(255, 255, 255, 255, 0)
            self.wait(tempo_ligado)

            self.set_cor(255, 255, 255, 255, 255)

```

```
self.wait(tempo_ligado)
```

```
self.set_cor(125, 125, 125, 125, 125)  
self.wait(tempo_ligado)
```

```
self.set_cor(50, 50, 50, 50, 50)  
self.wait(tempo_ligado)  
self.set_cor(200, 200, 200, 200, 200)  
self.wait(tempo_ligado)
```

```
for a in range (2):#
```

```
    #1
```

```
    self.set_cor(255, 255, 0, 0, 0)  
    self.wait(tempo_ligado)  
    self.set_cor(0, 0, 255, 255, 255)  
    self.wait(tempo_ligado)
```

```
    self.set_cor(255, 0, 0, 0, 255)  
    self.wait(tempo_ligado)  
    self.set_cor(255, 255, 255, 255, 0)  
    self.wait(tempo_ligado)
```

```
    self.set_cor(0, 255, 255, 255, 0)  
    self.wait(tempo_ligado)
```

```
    self.set_cor(255, 0, 255, 0, 255)  
    self.wait(tempo_ligado)
```

```
    self.set_cor(0, 255, 0, 255, 0)  
    self.wait(tempo_ligado)  
    self.set_cor(200, 200, 200, 200, 200)  
    self.wait(tempo_ligado)
```

```
for a in range (2):#
```

```
    #1
```

```
    self.set_cor(100, 100, 0, 0, 0)  
    self.wait(tempo_ligado)  
    self.set_cor(0, 0, 100, 100, 100)  
    self.wait(tempo_ligado)
```

```
    self.set_cor(100, 0, 0, 0, 100)  
    self.wait(tempo_ligado)  
    self.set_cor(100, 100, 100, 100, 0)  
    self.wait(tempo_ligado)
```

```
    self.set_cor(0, 100, 100, 100, 0)  
    self.wait(tempo_ligado)
```

```

self.set_cor(100, 0, 100, 0, 100)
self.wait(tempo_ligado)

self.set_cor(0, 100, 0, 100, 0)
self.wait(tempo_ligado)
self.set_cor(150, 150, 150, 150, 150)
self.wait(tempo_ligado)
for a in range (2):#
    #1
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(255, 0, 255, 255, 255)
    self.wait(tempo_ligado)

    self.set_cor(255, 0, 0, 0, 0)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_ligado)

    self.set_cor(255, 255, 255, 0, 0)
    self.wait(tempo_ligado)

    self.set_cor(255, 255, 255, 255, 0)
    self.wait(tempo_ligado)

    self.set_cor(255, 255, 255, 255, 255)
    self.wait(tempo_ligado)
    self.set_cor(150, 150, 150, 150, 150)
    self.wait(tempo_ligado)

def sequencia_avaba(self, duracao_total_ms=240000):
    """Sequência para Baile de Favela"""
    print("🕺 Iniciando sequência Baile de Favela...")

    """Versão limpa: matriz de cores → executa uma por uma"""
    print("🎵 Lady Gaga - Versão Limpa")

    # CONFIGURAÇÃO
    tempo_ligado = 800 # ms com luz acesa
    tempo_desligado = 800 # ms com luz apagada

    # MATRIZ DE CORES - uma para cada batida
    # Formato: (R, G, B, W, Y)

    for a in range (25):

```

```

#1
self.set_cor(255, 255, 255, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 0, 0, 255, 255)
self.wait(tempo_desligado)
#40s
self.set_cor(0, 0, 0, 0, 0)
self.wait(10000)
#50s
self.set_cor(125, 125, 125, 125, 125)
self.wait(5000)
#55s
tempo_ligado = 250 # ms com luz acesa
tempo_desligado = 250 # ms com luz apagada

```

for a in range (1):

```

#1
self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 255, 0, 0, 0)
self.wait(tempo_desligado)
self.set_cor(0, 0, 255, 0, 0)
self.wait(tempo_desligado)
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_desligado)

self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(255, 255, 0, 0, 0)
self.wait(tempo_desligado)
self.set_cor(255, 255, 255, 0, 0)
self.wait(tempo_desligado)
self.set_cor(255, 255, 255, 255, 0)
self.wait(tempo_desligado)
#57s
tempo_ligado = 125 # ms com luz acesa
tempo_desligado = 125 # ms com luz apagada

```

for a in range (1):

```

#1
self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(0, 255, 0, 0, 0)
self.wait(tempo_desligado)
self.set_cor(0, 0, 255, 0, 0)
self.wait(tempo_desligado)
self.set_cor(0, 0, 0, 255, 0)
self.wait(tempo_desligado)

```

```

self.set_cor(255, 0, 0, 0, 0)
self.wait(tempo_ligado)
self.set_cor(255, 255, 0, 0, 0)
self.wait(tempo_desligado)
self.set_cor(255, 255, 255, 0, 0)
self.wait(tempo_desligado)
self.set_cor(255, 255, 255, 255, 0)
self.wait(tempo_desligado)
#58s
tempo_ligado = 62 # ms com luz acesa
tempo_desligado = 63 # ms com luz apagada

for a in range (56):
    #1
    self.set_cor(0, 0, 125, 125, 125)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 0, 0, 0)
    self.wait(tempo_desligado)
    self.set_cor(0, 0, 125, 125, 125)
    self.wait(tempo_ligado)
    self.set_cor(125, 125, 0, 0, 0)
    self.wait(tempo_desligado)

    self.set_cor(125, 0, 0, 0, 125)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
    self.set_cor(125, 0, 0, 0, 125)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
#1min 26s
self.set_cor(0, 0, 0, 0, 0)
self.wait(4000)
#1min 30s
self.set_cor(100, 100, 100, 100, 100)
self.wait(30000)
#2min
self.set_cor(0, 0, 0, 0, 0)
self.wait(5000)
#2min 5s
self.fade(255, 255, 255, 255, 255, 5000)
#2min 10s
self.wait(3000)
#2min 13s
tempo_ligado = 750 # ms com luz acesa
tempo_desligado = 750 # ms com luz apagada

```

```

for a in range (2):
    #1
    self.set_cor(0, 0, 255, 255, 255)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_desligado)
    self.set_cor(0, 0, 255, 255, 255)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_desligado)

    self.set_cor(255, 0, 0, 0, 255)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
    self.set_cor(255, 0, 0, 0, 255)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
tempo_ligado = 125 # ms com luz acesa
tempo_desligado = 125 # ms com luz apagada
#2min 25s
for a in range (15):
    #1
    self.set_cor(0, 0, 255, 255, 255)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_desligado)
    self.set_cor(0, 0, 255, 255, 255)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_desligado)

    self.set_cor(255, 0, 0, 0, 255)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
    self.set_cor(255, 0, 0, 0, 255)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
#2min 40s
tempo_ligado = 25 # ms com luz acesa
tempo_desligado = 25 # ms com luz apagada

for a in range (150):
    #1

```



```

self.set_cor(0, 0, 255, 255, 255)
self.wait(tempo_ligado)
self.set_cor(255, 255, 0, 0, 0)
self.wait(tempo_desligado)
self.set_cor(0, 0, 255, 255, 255)
self.wait(tempo_ligado)
self.set_cor(255, 255, 0, 0, 0)
self.wait(tempo_desligado)

self.set_cor(255, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(0, 255, 255, 255, 0)
self.wait(tempo_desligado)
self.set_cor(255, 0, 0, 0, 255)
self.wait(tempo_ligado)
self.set_cor(0, 255, 255, 255, 0)
self.wait(tempo_desligado)
#3min 10s
tempo_ligado = 750 # ms com luz acesa
tempo_desligado = 750 # ms com luz apagada

```

```

for a in range (11):

```

```

    #1
    self.set_cor(0, 0, 255, 255, 255)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_desligado)
    self.set_cor(0, 0, 255, 255, 255)
    self.wait(tempo_ligado)
    self.set_cor(255, 255, 0, 0, 0)
    self.wait(tempo_desligado)

    self.set_cor(255, 0, 0, 0, 255)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
    self.set_cor(255, 0, 0, 0, 255)
    self.wait(tempo_ligado)
    self.set_cor(0, 255, 255, 255, 0)
    self.wait(tempo_desligado)
#3min 50s
self.set_cor(255, 255, 255, 255, 255)
self.wait(5000)
self.set_cor(0, 0, 0, 0, 0)
return True

```

```

# =====
# CONTROLE COM MÚSICA

```

```
# =====
```

```
def sequencia_baile_favela(self, duracao_total_ms=30000):
```

```
    """Sequência para Baile de Favela"""
```

```
    print("🕺 Iniciando sequência Baile de Favela...")
```

```
    """Versão limpa: matriz de cores → executa uma por uma"""
```

```
    print("🎵 Lady Gaga - Versão Limpa")
```

```
    # CONFIGURAÇÃO
```

```
    tempo_ligado = 1000 # ms com luz acesa
```

```
    tempo_desligado = 1000 # ms com luz apagada
```

```
    # MATRIZ DE CORES - uma para cada batida
```

```
    # Formato: (R, G, B, W, Y)
```

```
    self.wait(400)
```

```
    for a in range(60):
```

```
        #1
```

```
        self.set_cor(255, 255, 255, 0, 0)
```

```
        self.wait(tempo_ligado)
```

```
        self.set_cor(0, 0, 0, 255, 255)
```

```
        self.wait(tempo_desligado)
```

```
    return True
```

```
# =====
```

```
# CONTROLE COM MÚSICA
```

```
# =====
```

```
def executar_show_com_musica(self, nome_musica, sequencia_func,
```

```
    duracao_show_segundos=None):
```

```
    """
```

```
    Executa música e sequência de luzes sincronizadas
```

```
    Args:
```

```
        nome_musica: Nome da música no dicionário MUSICAS
```

```
        sequencia_func: Função de sequência de luzes
```

```
        duracao_show_segundos: Duração do show (None = música completa)
```

```
    """
```

```
    # Verifica música
```

```
    if nome_musica not in MUSICAS:
```

```
        print(f"❌ Música não encontrada: {nome_musica}")
```

```
        print("Músicas disponíveis:", list(MUSICAS.keys()))
```

```
        return False
```

```
    caminho_musica = Path(MUSICAS[nome_musica])
```

```

if not caminho_musica.exists():
    print(f"❌ Arquivo não encontrado: {caminho_musica}")
    return False

# Inicializa pygame mixer
try:
    pygame.mixer.init()
    pygame.mixer.music.load(str(caminho_musica))
except Exception as e:
    print(f"❌ Erro ao carregar música: {e}")
    return False

print("\n" + "=" * 60)
print(f"    SHOW: {nome_musica.upper().replace('_', ' ')}")
print("=" * 60)

# Thread para as luzes
def executar_luzes():
    try:
        if duracao_show_segundos:
            duracao_ms = duracao_show_segundos * 1000
            sequencia_func(duracao_ms)
        else:
            # Executa até a música terminar
            # (função precisa ter lógica de parada)
            sequencia_func(60000) # 1 minuto mínimo
    except Exception as e:
        print(f"❌ Erro na sequência de luzes: {e}")

# Contagem regressiva
print("🎵 Pronto para iniciar...")
for i in range(3, 0, -1):
    print(f"  {i}...")
    time.sleep(1)

print(" 🎵 GO! 🎵")

# Inicia luzes e música simultaneamente
thread_luzes = threading.Thread(target=executar_luzes)
thread_luzes.start()
pygame.mixer.music.play()

# Monitora execução
try:
    if duracao_show_segundos:
        # Show com duração fixa
        time.sleep(duracao_show_segundos)
        pygame.mixer.music.stop()

```

```

else:
    # Aguarda música terminar naturalmente
    while pygame.mixer.music.get_busy():
        time.sleep(0.1)

    # Aguarda thread de luzes terminar
    thread_luzes.join(timeout=5)

except KeyboardInterrupt:
    print("\n ⚠ Show interrompido pelo usuário")
    pygame.mixer.music.stop()

finally:
    # Garante que tudo está limpo
    self.reset()
    if thread_luzes.is_alive():
        print(" 🟡 Finalizando sequência de luzes...")

    pygame.mixer.music.stop()
    pygame.mixer.quit()

print("\n" + "=" * 60)
print("      SHOW FINALIZADO!")
print("=" * 60)

return True

# =====
# FUNÇÕES UTILITÁRIAS
# =====

def testar_todas_cores(self):
    """Teste rápido de todas as cores"""
    print(" 🎨 Testando todas as cores...")

    testes = [
        ("Vermelho", (255, 0, 0, 0, 0)),
        ("Verde", (0, 255, 0, 0, 0)),
        ("Azul", (0, 0, 255, 0, 0)),
        ("Branco", (0, 0, 0, 255, 0)),
        ("Amarelo", (0, 0, 0, 0, 255)),
        ("Rosa", (255, 0, 255, 100, 0)),
        ("Ciano", (0, 255, 255, 100, 0)),
        ("Laranja", (255, 165, 0, 0, 100)),
        ("Roxo", (128, 0, 128, 50, 0)),
    ]

    for nome, cor in testes:

```

```

        print(f" {nome}...")
        self.set_cor(*cor)
        time.sleep(0.5)

    self.reset()
    print("✅ Teste de cores completo!")

def modo_demo(self):
    """Modo demo automático"""
    print("🚀 Iniciando modo demo...")

    demos = [
        ("Lady Gaga", self.sequencia_lady_gaga_simples, 30),
        ("Baile de Favela", self.sequencia_baile_favela, 30),
        ("avaba", self.sequencia_avaba, 30),
    ]

    for nome, sequencia, duracao in demos:
        print(f"\n📺 Demo: {nome} ({duracao}s)")
        input(" Pressione Enter para começar...")

        sequencia(duracao * 1000)

        print(f"✅ Demo {nome} completo!")
        time.sleep(1)

    print("\n🎉 Todas as demos finalizadas!")

def desconectar(self):
    """Desconecta do ESP32"""
    if self.conexao:
        self.reset()
        self.conexao.close()
        self.conectado = False
        print("🔌 Desconectado do ESP32")

# =====
# FUNÇÃO PRINCIPAL - INTERFACE DE USUÁRIO
# =====
def main():
    """Interface principal do programa"""

    print("=" * 60)
    print("    SHOW DE LUZES ESP32 - CONTROLE DEFINITIVO")
    print("=" * 60)

    # Verificar pygame
    try:

```

```

import pygame
except ImportError:
    print("❌ Pygame não está instalado.")
    print("  Instale com: pip install pygame")
    sys.exit(1)

# Criar controlador
controller = ESP32ShowController(PORTA_COM, BAUD_RATE)

# Conectar
if not controller.conectar():
    print("❌ Não foi possível conectar ao ESP32")
    return

try:
    while True:
        print("\n" + "=" * 40)
        print("MENU PRINCIPAL")
        print("=" * 40)
        print("1. 🎵 Lady Gaga - Show completo")
        print("2. 🕺 Baile de Favela - Show completo")
        print("3. 🎨 Teste rápido de cores")
        print("4. 🚀 Modo demo automático")
        print("5. 🎮 Controle manual")
        print("6. 📊 Status do sistema")
        print("7. ❌ Sair")
        print("=" * 40)

        escolha = input("\nEscolha uma opção (1-7): ").strip()

        if escolha == "1":
            # Lady Gaga
            controller.executar_show_com_musica(
                "lady_gaga",
                controller.sequencia_lady_gaga_simples,
                duracao_show_segundos=120 # 30 segundos para teste
            )

        elif escolha == "2":
            # Baile de Favela
            controller.executar_show_com_musica(
                "baile_favela",
                controller.sequencia_baile_favela,
                duracao_show_segundos=120
            )

        elif escolha == "11":
            # Baile de Favela
            controller.executar_show_com_musica(

```

```
    "avaba",
    controller.sequencia_avaba,
    duracao_show_segundos=240
)
```

```
elif escolha == "3":
    # Teste de cores
    controller.testar_todas_cores()
```

```
elif escolha == "4":
    # Modo demo
    controller.moda_demo()
```

```
elif escolha == "5":
    # Controle manual
    print("\n🕹️ Controle Manual")
    print("Formato: R G B W Y (0-255)")
    print("Exemplo: '255 0 0 0 0' para vermelho")
    print("Digite 'voltar' para retornar")
```

```
while True:
    entrada = input("\nCor (R G B W Y): ").strip().lower()
```

```
    if entrada == "voltar":
        break
```

```
    try:
        valores = [int(v) for v in entrada.split()]
        if len(valores) == 5:
            if controller.set_cor(*valores):
                print(f"✅ Cor definida: {valores}")
            else:
                print(f"❌ Falha ao definir cor")
        else:
            print(f"❌ Precisa de 5 valores")
    except ValueError:
        print(f"❌ Valores inválidos")
    except Exception as e:
        print(f"❌ Erro: {e}")
```

```
elif escolha == "6":
    # Status
    print("\n📊 Status do Sistema:")
    print(f"  Porta: {PORTA_COM}")
    print(f"  Conectado: {'✅' if controller.conectado else '❌'}")
    print(f"  Cor atual: {controller.cor_atual}")
    print(f"  Músicas configuradas: {len(MUSICAS)}")
```

```
elif escolha == "7":
```

```
    # Sair
```

```
    print("\n👋 Saindo...")
```

```
    break
```

```
elif escolha == "8":
```

```
    # Sair
```

```
    print("\n👋 Saindo...")
```

```
# =====  
# SHOW SEQUENCIAL MANUAL (SEM MENU)  
# =====
```

```
REPETICOES = 5    # quantas vezes repetir o ciclo completo
```

```
PAUSA_ENTRE = 60    # segundos entre músicas
```

```
print("\n🔄 Iniciando show sequencial manual...")
```

```
print(f"🔄 Repetições: {REPETICOES}")
```

```
print("-" * 50)
```

```
for ciclo in range(REPETICOES):
```

```
    print(f"\n🎬 CICLO {ciclo + 1}/{REPETICOES}")
```

```
    # -----
```

```
    # 1 LADY GAGA
```

```
    # -----
```

```
    print("🎵 Tocando: dont stop")
```

```
    controller.executar_show_com_musica(
```

```
        "dont_stop",
```

```
        controller.sequencia_dont_stop_simples,
```

```
        duracao_show_segundos=240
```

```
    )
```

```
    print("⏸ Pausa...")
```

```
    time.sleep(PAUSA_ENTRE)
```

```
    # -----
```

```
    # 2 BAILE DE FAVELA
```

```
    # -----
```

```
    print("🎵 Tocando: Baile de Favela")
```

```
    controller.executar_show_com_musica(
```

```
        "baile_favela",
```

```
        controller.sequencia_baile_favela,
```

```
        duracao_show_segundos=None
```

```
    )
```



```
print("⏸ Pausa...")
time.sleep(PAUSA_ENTRE)
time.sleep(PAUSA_ENTRE)
```

```
# -----
```

```
# 3 avaba
```

```
# -----
```

```
print("🎵 Tocando: Baile de Favela")
controller.executar_show_com_musica(
    "avaba",
    controller.sequencia_avaba,
    duracao_show_segundos=240
)
```

```
print("⏸ Pausa...")
time.sleep(PAUSA_ENTRE)
```

```
print("⏸ Pausa...")
time.sleep(PAUSA_ENTRE)
```

```
# -----
```

```
# 2 BAILE DE FAVELA
```

```
# -----
```

```
print("🎵 Tocando: lady gaga")
```

```
controller.executar_show_com_musica(
    "lady_gaga",
    controller.sequencia_lady_gaga_simples,
    duracao_show_segundos=None # toca até acabar
)
```

```
print("⏸ Pausa...")
time.sleep(PAUSA_ENTRE)
```

```
print("\n🎉 Show sequencial finalizado!")
controller.reset()
```

```
break
```

```
else:
```

```
print("❌ Opção inválida")
```

```

except KeyboardInterrupt:
    print("\n\n ⚠ Programa interrompido pelo usuário")
except Exception as e:
    print(f"\n ❌ Erro inesperado: {e}")
finally:
    controller.desconectar()
    print("\n" + "=" * 60)
    print("PROGRAMA FINALIZADO")
    print("=" * 60)

```

```

# =====
# SEQUÊNCIAS PERSONALIZADAS (ADICIONE AQUI!)
# =====

```

```

def criar_sequencia_personalizada():
    """
    Template para criar suas próprias sequências.
    Copie e modifique esta função.
    """

    def minha_sequencia(controller, duracao_ms):
        """Minha sequência personalizada"""

        # Configuração
        tempo = 250

        # Suas cores personalizadas
        minhas_cores = [
            (255, 0, 0, 0, 0),    # Vermelho
            (0, 255, 0, 0, 0),    # Verde
            # Adicione mais cores...
        ]

        # Calcula repetições
        repeticoes = duracao_ms // (tempo * 2)

        for i in range(repeticoes):
            cor_idx = i % len(minhas_cores)
            r, g, b, w, y = minhas_cores[cor_idx]

            # Padrão: liga → espera → desliga → espera
            controller.set_cor(r, g, b, w, y)

```

```

        controller.wait(tempo)
        controller.set_cor(0, 0, 0, 0, 0)
        controller.wait(tempo)

    return True

return minha_sequencia

# =====
# EXECUÇÃO
# =====
if __name__ == "__main__":
    # Verificar caminhos das músicas
    print("🔍 Verificando arquivos de música...")
    for nome, caminho in MUSICAS.items():
        if Path(caminho).exists():
            print(f"✅ {nome}: OK")
        else:
            print(f"❌ {nome}: NÃO ENCONTRADO")
            print(f"    {caminho}")

# Executar programa principal
main()

```

ESP32

```

#include <Arduino.h>

// =====
// DEFINIÇÕES DE PINOS E CANAIS
// =====

#define PINO_VERMELHO 14
#define PINO_VERDE 12
#define PINO_AZUL 13
#define PINO_BRANCO 27
#define PINO_AMARELO 26

#define CANAL_VERMELHO 0
#define CANAL_VERDE 1
#define CANAL_AZUL 2
#define CANAL_BRANCO 3
#define CANAL_AMARELO 4

```

```

#define PWM_FREQ          1000
#define PWM_RESOLUCAO     8

// =====
// VARIÁVEIS GLOBAIS (ADICIONE ESTA SEÇÃO!)
// =====
String comandoBuffer = ""; // ADICIONE ESTA LINHA
unsigned long ultimoComandoTime = 0;
bool executandoSequencia = false;
bool modo_confirmacao = true;

// Armazena cor atual para fades precisos
int cor_atual[5] = {0, 0, 0, 0, 0};

// =====
// CONFIGURAÇÃO PWM
// =====
void configurarPWM() {
    ledcSetup(CANAL_VERMELHO, PWM_FREQ, PWM_RESOLUCAO);
    ledcSetup(CANAL_VERDE, PWM_FREQ, PWM_RESOLUCAO);
    ledcSetup(CANAL_AZUL, PWM_FREQ, PWM_RESOLUCAO);
    ledcSetup(CANAL_BRANCO, PWM_FREQ, PWM_RESOLUCAO);
    ledcSetup(CANAL_AMARELO, PWM_FREQ, PWM_RESOLUCAO);

    ledcAttachPin(PINO_VERMELHO, CANAL_VERMELHO);
    ledcAttachPin(PINO_VERDE, CANAL_VERDE);
    ledcAttachPin(PINO_AZUL, CANAL_AZUL);
    ledcAttachPin(PINO_BRANCO, CANAL_BRANCO);
    ledcAttachPin(PINO_AMARELO, CANAL_AMARELO);
}

// =====
// FUNÇÕES DE CONTROLE DE COR
// =====
void setCor(int verm, int verd, int azul, int bran, int amar) {
    ledcWrite(CANAL_VERMELHO, verm);
    ledcWrite(CANAL_VERDE, verd);
    ledcWrite(CANAL_AZUL, azul);
    ledcWrite(CANAL_BRANCO, bran);
    ledcWrite(CANAL_AMARELO, amar);

    // Atualiza cor atual
    cor_atual[0] = verm;

```

```

    cor_atual[1] = verd;
    cor_atual[2] = azul;
    cor_atual[3] = bran;
    cor_atual[4] = amar;
}

// =====
// FUNÇÃO PARA EXTRAIR PARÂMETROS (ADICIONE!)
// =====
int extrairParametros(String paramStr, int* values, int maxParams) {
    int count = 0;
    int startIndex = 0;

    while (count < maxParams && startIndex < paramStr.length()) {
        int endIndex = paramStr.indexOf(',', startIndex);
        if (endIndex == -1) endIndex = paramStr.length();

        String param = paramStr.substring(startIndex, endIndex);
        param.trim();

        if (param.length() > 0) {
            values[count] = param.toInt();
            count++;
        }

        startIndex = endIndex + 1;
    }

    return count;
}

// =====
// FUNÇÃO DE FADE
// =====
void fadeCor(int r_final, int g_final, int b_final, int w_final, int
y_final, int tempo_ms) {
    int passos = max(1, tempo_ms / 10);

    for (int i = 0; i <= passos; i++) {
        float progresso = (float)i / passos;

        int r = cor_atual[0] + (r_final - cor_atual[0]) * progresso;
        int g = cor_atual[1] + (g_final - cor_atual[1]) * progresso;

```

```

        int b = cor_atual[2] + (b_final - cor_atual[2]) * progresso;
        int w = cor_atual[3] + (w_final - cor_atual[3]) * progresso;
        int y = cor_atual[4] + (y_final - cor_atual[4]) * progresso;

        setCor(r, g, b, w, y);
        delay(tempo_ms / passos);
    }
}

// =====
// FUNÇÕES DE CONFIRMAÇÃO
// =====
void enviarConfirmacao(String tipo, String detalhe = "") {
    if (modo_confirmacao) {
        if (detalhe.length() > 0) {
            Serial.println("OK:" + tipo + ":" + detalhe);
        } else {
            Serial.println("OK:" + tipo);
        }
    }
}

void enviarErro(String tipo, String detalhe = "") {
    if (detalhe.length() > 0) {
        Serial.println("ERR:" + tipo + ":" + detalhe);
    } else {
        Serial.println("ERR:" + tipo);
    }
}

// =====
// PROCESSAMENTO DE COMANDOS
// =====
void processarComando(String cmd) {
    cmd.trim();
    if (cmd.length() == 0) return;

    // Echo para debug
    if (modo_confirmacao) {
        Serial.print("RX:");
        Serial.println(cmd);
    }
}

```

```

int separatorIndex = cmd.indexOf(',');
String comando = cmd;
String params = "";

if (separatorIndex != -1) {
    comando = cmd.substring(0, separatorIndex);
    params = cmd.substring(separatorIndex + 1);
}

comando.toUpperCase();

// COMANDOS DE CONFIGURAÇÃO
if (comando == "ACK") {
    if (params == "1" || params == "ON") {
        modo_confirmacao = true;
        enviarConfirmacao("ACK", "ON");
    } else {
        modo_confirmacao = false;
        enviarConfirmacao("ACK", "OFF");
    }
    return;
}

else if (comando == "STATUS") {
    Serial.print("STATUS:");
    Serial.print("CONNECTED,");
    Serial.print("ACK:");
    Serial.print(modo_confirmacao ? "ON" : "OFF");
    Serial.print(",TIME:");
    Serial.println(millis());
    return;
}

// COMANDOS DE CONTROLE
if (comando == "S") {
    int values[5];
    int paramCount = extrairParametros(params, values, 5);

    if (paramCount == 5) {
        setCor(values[0], values[1], values[2], values[3], values[4]);
        enviarConfirmacao("SET", String(millis()));
    } else {
        enviarErro("PARAMS", "SET precisa de 5 valores");
    }
}

```

```

    }
}

else if (comando == "W") {
    int values[1];
    if (extrairParametros(params, values, 1) == 1) {
        unsigned long waitStart = millis();
        delay(values[0]);
        unsigned long waitEnd = millis();
        enviarConfirmacao("WAIT", String(waitEnd - waitStart));
    } else {
        enviarErro("PARAMS", "WAIT precisa de 1 valor");
    }
}

else if (comando == "F") {
    int values[6];
    if (extrairParametros(params, values, 6) == 6) {
        unsigned long fadeStart = millis();
        fadeCor(values[0], values[1], values[2], values[3], values[4],
values[5]);
        unsigned long fadeEnd = millis();
        enviarConfirmacao("FADE", String(fadeEnd - fadeStart));
    } else {
        enviarErro("PARAMS", "FADE precisa de 6 valores");
    }
}

else if (comando == "START") {
    executandoSequencia = true;
    enviarConfirmacao("START");
}

else if (comando == "STOP") {
    executandoSequencia = false;
    setCor(0, 0, 0, 0, 0);
    enviarConfirmacao("STOP");
}

else if (comando == "RESET") {
    setCor(0, 0, 0, 0, 0);
    enviarConfirmacao("RESET");
}

```



```

else if (comando == "PING") {
    enviarConfirmacao("PONG", String(millis()));
}

else {
    enviarErro("UNKNOWN", comando);
}

ultimoComandoTime = millis();
}

// =====
// SETUP
// =====
void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    configurarPWM();
    setCor(0, 0, 0, 0, 0);

    Serial.println("\n\n=== ESP32 LED CONTROLLER ===");
    Serial.println("Modo: ESCRAVO COM CONFIRMAÇÃO");
    Serial.println("Aguardando comandos...");
    Serial.println("Comandos:");
    Serial.println("  S,r,g,b,w,y      - Set color");
    Serial.println("  W,ms             - Wait");
    Serial.println("  F,r,g,b,w,y,ms   - Fade");
    Serial.println("  ACK,0/1           - Liga/desliga confirmações");
    Serial.println("  STATUS            - Status do sistema");
    Serial.println("  PING              - Teste de conexão");
    Serial.println("=====");
    Serial.println("READY:ACK_ON");
}

// =====
// LOOP PRINCIPAL
// =====
void loop() {
    // Leitura de comandos da Serial

```

```

while (Serial.available()) {
    char c = Serial.read();

    if (c == '\n') {
        processarComando(comandoBuffer);
        comandoBuffer = "";
    } else {
        comandoBuffer += c;
    }
}

// Timeout de segurança
if (comandoBuffer.length() > 100) {
    if (modo_confirmacao) {
        Serial.println("ERR:BUFFER_OVERFLOW");
    }
    comandoBuffer = "";
}

// Heartbeat a cada 10 segundos
static unsigned long lastHeartbeat = 0;
if (millis() - lastHeartbeat > 10000) {
    lastHeartbeat = millis();
    if (modo_confirmacao) {
        Serial.print("HEARTBEAT:");
        Serial.println(millis());
    }
}

// Execução de sequência (se estiver ativa)
if (executandoSequencia) {
    // Aqui você pode colocar sequências locais se quiser
    // Mas o controle principal está no Python
    delay(100);
}
}

```

Observações sobre os códigos

Eles possuem uma espécie de confirmação a cada comando do Python enviado ao ESP32.

O ESP32 é o escravo e o Python é o mestre. O ESP32 apenas recebe ordens de quais pinos ele deve ligar. Só que existe uma confirmação necessária para que o python envie novos comandos, o que pode tornar um pouco mais lenta a piscagem. Só que essa lentidão é importante apenas em momentos em que forem mais frequentes os erros de comunicação. No geral, isso não importa muito.

Eu escolhi as músicas abaixo:

```
"baile_favela": r"C:\Users\eduar\Downloads\Bobby Helms - Jingle Bell Rock.mp3",  
"lady_gaga": r"C:\Users\eduar\Downloads\ladygaga.mp3",  
"avaba": r"C:\Users\eduar\Downloads\25 MINUTOS DE ELETRÔNICAS PESADAS [ 150  
BPM ] DJ RANTARO 2K18.mp3",  
"dont_stop": r"C:\Users\eduar\Downloads\Journey - Don't Stop Believin' (Escape Tour  
1981_ Live In Houston).mp3"
```

elas devem ser salvas na mesma pasta do código, com esses nomes.

Aquela “Jingle Bell Rock” tem uma piscada por segundo, então é de boassa. Aquela “avaba” tem um determinado número de piscagens por segundo, que é simples de reproduzir, não lembro se era 120 BPM (2 por segundo) ou 150, o mesmo vale para a música “Don't Stop Believin”.

Aquela que eu chamei de “avaba” eu pus alguns efeitos diferentes conforme o passar da música, para se adequar a ela.

Código extra em python para limpar a memória caso haja muita transferência de dados e lixo de memória

```
import serial  
import time  
  
PORTA_COM = "COM6"  
BAUD_RATE = 115200  
  
def limpar_buffer_esp32():  
    print(f"🔌 Abrindo {PORTA_COM}...")  
  
    ser = serial.Serial(  
        port=PORTA_COM,  
        baudrate=BAUD_RATE,  
        timeout=1,  
        dsrdtr=False,  
        rtscts=False  
    )
```

```

# Evita reset por DTR/RTS
ser.dtr = False
ser.rts = False

# ESP32 geralmente reseta ao abrir a porta
print("⌚ Aguardando boot do ESP32...")
time.sleep(2.5)

# Limpeza pesada de buffers
ser.reset_input_buffer()
ser.reset_output_buffer()

time.sleep(0.5)
ser.reset_input_buffer()

print("🧹 Buffer do PC limpo")

# Ler e descartar qualquer lixo que ainda chegue
inicio = time.time()
lixo = []

while time.time() - inicio < 2:
    if ser.in_waiting:
        linha = ser.readline().decode(errors="ignore").strip()
        if linha:
            lixo.append(linha)

if lixo:
    print("🗑 Lixo descartado do ESP32:")
    for l in lixo:
        print("  >", l)
else:
    print("✅ Nenhum lixo recebido")

# Handshake simples (opcional, mas recomendado)
print("🔗 Sincronizando...")
sincronizado = False

for _ in range(5):
    ser.write(b"PING\n")
    time.sleep(0.2)

    if ser.in_waiting:
        resp = ser.readline().decode(errors="ignore").strip()
        if "PONG" in resp or "OK" in resp or "READY" in resp:
            print(f"✅ ESP32 respondeu: {resp}")
            sincronizado = True

```

```
break
```

```
if not sincronizado:
```

```
    print("⚠ ESP32 não respondeu ao PING, mas buffer está limpo")
```

```
print("✅ Comunicação limpa e pronta")
```

```
return ser
```

```
if __name__ == "__main__":
```

```
    ser = limpar_buffer_esp32()
```

```
    # Teste opcional
```

```
    print("✍ Enviando teste...")
```

```
    ser.write(b"S,0,0,0,0,0\n")
```

```
    time.sleep(0.5)
```

```
    ser.close()
```

```
    print("🔊 Porta fechada")
```